

Interfaces Advanced Features

1. Default Interface Methods (C# 8.0+)

Introduction

- Traditionally purely abstract, interfaces gained implementation capability in C# 8.0 (.NET Core 3.0, 2019) to support API evolution without breaking changes.

Key Characteristics

- Backward Compatibility:** Add functionality without forcing implementation
- Multiple Inheritance:** Classes can inherit multiple default implementations
- Explicit Invocation:** Default methods must be called through interface reference

```
public interface ILogger {
    void Log(string message); // Traditional abstract method

    // Default implementation
    void LogError(string error) {
        Log($"ERROR: {error}");
    }
}

class ConsoleLogger : ILogger
{
    public void Log(string message) => Console.WriteLine(message);
    // No need to implement LogError
}

// Usage:
ILogger logger = new ConsoleLogger();
logger.LogError("Something failed"); // Uses default implementation
```

2. Static Abstract Members (C# 11+)

Generic Math Support

- Enables abstract static methods in interfaces, primarily for numeric scenarios:

```
public interface IAddable<T> where T : IAddable<T>
{
    static abstract T operator +(T left, T right);
    static virtual T Zero => default;
}

public struct Point : IAddable<Point>
{
}
```

```
public int X, Y;

public static Point operator +(Point left, Point right) =>
    new Point { X = left.X + right.X, Y = left.Y + right.Y };

public static Point Zero => new Point();
}
```

Key Benefits

1. **Type-Safe Operator Overloading**
2. **Generic Algorithm Support**
3. **Numerics Without Boxing**

3. Version-Resilient Interfaces

1. Add new methods as defaults
2. Never remove existing methods
3. Use extension methods for utility functions

4. Interface Full Member Support

- Interfaces can declare all member types:

```
public interface IObservable {
    event EventHandler Changed;
    string Name { get; set; }
    int Id { get; }
}
```

- All these must be implemented in derived classes.

5. Diamond Inheritance Resolution

- Default methods handle multiple inheritance cases:
- Example1:

```
class Intf1 {
    public void Fun() {
        Console.WriteLine("Intf1.Fun() called.");
    }
}

class Intf2 {
    public void Fun() {
        Console.WriteLine("Intf2.Fun() called.");
    }
}
```

```
class MyClass : Intf1, Intf2 {  
}  
  
// Usage : e.g. in Main()  
Intf1 obj1 = new MyClass();  
obj1.Fun(); // Intf1.Fun() called.  
  
MyClass obj = new MyClass();  
obj.Fun(); // Compiler error
```

- Example2:

```
interface IA { void M() => Console.Write("A"); }  
interface IB : IA { void IA.M() => Console.Write("B"); }  
interface IC : IA { void IA.M() => Console.Write("C"); }  
class D : IB, IC  
{  
    // Must provide implementation to resolve ambiguity - Otherwise compiler  
    error  
    void IA.M() => Console.Write("D");  
}
```

5. Best Practices

1. **Small, Focused Interfaces (ISP)**
2. **Default Methods for Backward Compatibility**
3. **Explicit Implementation for Clarity**
4. **Avoid State in Interfaces**
5. **Prefer Abstract Classes for Common Implementation**

MSDN References

- [Default Interface Methods](#)
- [Static Abstract Members](#)
- [Interface Design Guidelines](#)

dynamic Keyword

1. Introduction

- Introduced in **.NET Framework 4.0 (2010)** alongside C# 4.0 to simplify interoperability with **dynamic languages** (e.g., Python, JavaScript) and **COM objects**.
- Before **dynamic**, developers used **reflection** or explicit casting, which was verbose and error-prone.
- The **dynamic** keyword enables **late binding**, where type resolution happens at **runtime** instead of compile-time.
- Part of the **Dynamic Language Runtime (DLR)**, which sits atop the CLR to support dynamic operations.

2. Key Concepts & Definitions

- **Dynamic Typing:** The type of a `dynamic` variable is resolved at runtime, bypassing compile-time checks.
- **RuntimeBinder:** The component that resolves member invocations on `dynamic` objects (throws `RuntimeBinderException` on failures).
- **IDynamicMetaObjectProvider:** Interface used for custom dynamic behavior (e.g., `ExpandoObject`, `DynamicObject`).
- **DLR (Dynamic Language Runtime):** A layer that provides services for dynamic languages in .NET (e.g., caching call sites).
- **Use Cases:**
 - COM interop (e.g., Microsoft Office automation).
 - Consuming REST APIs with unpredictable schemas.
 - Dynamic data structures (e.g., `ExpandoObject`).

3. Advantages

- **Flexibility:** Simplifies interaction with weakly-typed systems (e.g., JSON, COM).
- **Reduces Boilerplate:** Avoids complex reflection code.
- **Improves Readability:** Cleaner syntax for dynamic operations.

4. Disadvantages

- **No Compile-Time Safety:** Errors (e.g., missing methods) only surface at runtime.
- **Performance Overhead:** Dynamic dispatch is slower than static typing due to runtime resolution.
- **Tooling Limitations:** IDE features like IntelliSense don't work for `dynamic` types.

5. How `dynamic` Differs from `var` and `object`

- **`var`:** Still **statically typed** (type inferred at compile-time).
- **`object`:** Requires explicit casting and offers no dynamic dispatch.
- **`dynamic`:** Defers all type checks to runtime.

6. Examples

```
// Example 1: Basic dynamic usage
dynamic obj = GetExternalData(); // Could be JSON/COM
Console.WriteLine(obj.Name); // Resolved at runtime
```

```
// Example 2: ExpandoObject (dynamic dictionary)
dynamic person = new ExpandoObject();
person.Name = "Alice";
person.Age = 30;
```

```
// Example 3: COM Interop (e.g., Excel)
dynamic excel = Microsoft.Office.Interop.Excel;
excel.Application app = new excel.Application();
```

7. RuntimeBinderException

- Thrown when a member (method/property) doesn't exist at runtime.
- Example:

```
dynamic value = "hello";
value.Foo(); // RuntimeBinderException: 'string' does not contain 'Foo'
```

8. Performance Considerations

- Avoid `dynamic` in performance-critical paths (e.g., tight loops).
- Cache dynamic calls if repeated (e.g., via `Func<dynamic, object>` delegates).

MSDN References

- [dynamic \(C# Reference\)](#)
- [DynamicObject Class](#)
- [DLR Overview](#)

Reflection

1. Introduction

- Reflection provides the ability to inspect and interact with type information at runtime. This powerful feature enables:
 - Dynamic type discovery
 - Late binding
 - Runtime code analysis
 - Self-modifying applications

2. Core Concepts

2.1 Type Metadata

- Type metadata is binary information that describes your type (class, struct, enum, ...). It's stored alongside the compiled code (IL) in assemblies.
- **Type Metadata**
 - Name: The fully qualified name of the type.
 - Visibility: Whether the type is public, private, etc.
 - Base Class: The type that the current type inherits from.

- Interfaces: Interfaces implemented by the type.
- Members: Information about the methods, fields, properties, events, and nested types within the type.
- Attributes/Flags: Additional descriptive information about the type e.g. `IsAbstract`, `IsSealed`, etc.
- **Member Metadata**
 - Name: The name of the member (e.g., method name, field name).
 - Type: The data type of the member (e.g., `int`, `string`, a custom type).
 - Visibility: Whether the member is public, private, etc.
 - Attributes: Additional descriptive information about the member.
- This metadata is loaded at runtime (when assembly is loaded).

2.2 Type Discovery

- The `System.Type` object holds type metadata.
- It can be accessed in one of the following ways:

```
Type classType = typeof(ClassName);           // Compile-time known type
Type objType = someObject.GetType();           // Runtime type
Type typeByName = Type.GetType("System.Int32"); // By name
```

2.3 Assembly Inspection

- Information from current assembly can be accessed as follows:

```
Assembly assembly = Assembly.GetExecutingAssembly();
foreach (Type type in assembly.GetTypes())
    Console.WriteLine(type.FullName);
```

- An assembly can be loaded explicitly as follows:

```
Assembly assembly = Assembly.LoadFrom(@"AssemblyPath");
foreach (Type type in assembly.GetTypes())
    Console.WriteLine(type.FullName);
```

3. Member Inspection

3.1 Exploring Members

```
Type type = typeof(MyClass);

// Get all public methods
MethodInfo[] methods = type.GetMethods();
```

```
// Get specific property
PropertyInfo[] props = type.GetProperties();

// Get all fields
FieldInfo[] field = type.GetFields(BindingFlags.Public | BindingFlags.NonPublic |
BindingFlags.Instance | BindingFlags.Static);

// Get field including non-public
FieldInfo field = type.GetField("_internal", BindingFlags.NonPublic |
BindingFlags.Instance);
```

3.2 Dynamic Object Creation

```
object obj = Activator.CreateInstance(type);
```

3.2 Method Invocation

```
// static method invoked without object
MethodInfo method = typeof(Math).GetMethod("Max", new[] { typeof(int), typeof(int)
});
object result = method.Invoke(null, new object[] { 5, 10 }); // Static method call
```

```
// non-static methods need object (usually dynamically created) as first arg to
method.Invoke()
object result = method.Invoke(obj, new object[] { ... }); // Non-Static method
call
```

4. Performance Considerations

4.1 Caching Strategies

```
// Cache expensive reflection operations
private static readonly MethodInfo _toStringMethod = typeof(object)
    .GetMethod("ToString");

// Reuse cached method
string result = (string)_toStringMethod.Invoke(obj, null);
```

4.2 Alternatives

- **Compiled Expressions:** Faster than pure reflection

```
Func<object, string> toString = obj => obj.ToString(); // Faster alternative
```

MSDN References

- [Reflection in .NET](#)
- [Type Class](#)
- [Runtime Type Handling](#)

Attributes

1. Introduction to Attributes

- Attributes provide a powerful way to add metadata and declarative information to code elements.
 - Adding descriptive information to types and members
 - Changing runtime behavior through metadata
 - Enabling framework features (serialization, COM interop, etc.)

2. Core Attribute Concepts

2.1 Attribute Targets

- Attributes can be applied to various program elements:

```
[Serializable]           // Class
public class MyClass {
    [Obsolete("Use NewMethod instead")] // Method
    public void OldMethod() {}

    [JsonProperty("user_name")] // Property
    public string UserName { get; set; }

    [NonSerialized]       // Field
    public int Value;
}
```

2.2 Built-in Attribute Categories

1. **Serialization:** [Serializable], [DataContract]
2. **COM Interop:** [ComVisible], [Guid]
3. **Compiler Directives:** [Conditional], [Obsolete]
4. **Testing:** [TestMethod], [DataRow]
5. **Security:** [Authorize], [AllowAnonymous]

3. Creating Custom Attributes

3.1 Basic Attribute Definition

```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method,
                AllowMultiple = false,
                Inherited = true)]
public class AuditAttribute : Attribute
{
    public string Category { get; }
    public AuditLevel Level { get; set; } = AuditLevel.Info;

    public AuditAttribute(string category)
    {
        Category = category;
    }
}

public enum AuditLevel { Info, Warning, Error }
```

3.2 Attribute Usage Parameters

- **AttributeTargets:** Where the attribute can be applied
- **AllowMultiple:** Whether multiple instances are allowed
- **Inherited:** Whether derived classes inherit the attribute

4. Attribute Reflection

4.1 Reading Attributes at Runtime

```
Type type = typeof(MyClass);

// Get class-level attributes
var attrs = type.GetCustomAttributes(typeof(AuditAttribute), false);

// Get method-specific attributes
MethodInfo method = type.GetMethod("Process");
var methodAttrs = method.GetCustomAttributes<ObsoleteAttribute>();
```

4.2 Practical Example: API Documentation Generator

```
public class DocumentationAttribute:Attribute {
    public DocumentationAttribute(string desc) {
        Description = desc;
    }
    public string Description { get; set; }
}

[Documentation("Represents a customer entity")]
public class Customer {
```

```
[Documentation("Unique identifier")]
public int Id { get; set; }

[Documentation("Customer's full name")]
public string Name { get; set; }
}

public static void GenerateDocs(Type type) {
    var classDoc = type.GetCustomAttribute<DocumentationAttribute>();
    Console.WriteLine($"Class: {type.Name} - {classDoc?.Description}");

    foreach (var prop in type.GetProperties()) {
        var propDoc = prop.GetCustomAttribute<DocumentationAttribute>();
        Console.WriteLine($"  {prop.Name}: {propDoc?.Description}");
    }
}
```

MSDN References

- [Creating Custom Attributes](#)
- [Attribute Usage](#)
- [Retrieving Attributes](#)

Unsafe Code and P/Invoke

1. unsafe Code Basics

Concept

- The `unsafe` keyword allows pointer operations and direct memory access in C#. Requires compiling with `/unsafe` flag.

Minimal Example

```
unsafe void PointerDemo()
{
    int value = 10;
    int* ptr = &value; // Pointer declaration
    Console.WriteLine(*ptr); // Dereference pointer
}
```

2. fixed Keyword

Concept

- Pins managed objects in memory to prevent GC relocation during pointer operations.

Minimal Example

```
unsafe void FixedDemo()
{
    int[] numbers = { 10, 20, 30 };
    fixed (int* ptr = numbers)
    {
        Console.WriteLine(ptr[1]); // Access array via pointer
    }
}
```

3. P/Invoke (Platform Invoke)

Concept

- Calls native functions from System DLLs. Requires `DllImport` attribute.

Minimal Example

```
using System.Runtime.InteropServices;

class NativeMethods
{
    [DllImport("user32.dll")]
    public static extern int MessageBox(IntPtr hWnd, string text, string caption,
    int type);
}

// Usage:
NativeMethods.MessageBox(IntPtr.Zero, "Hello", "Message", 0);
```

Key Notes

1. Requirements:

- Enable "Allow unsafe code" in project settings
- Mark methods with `unsafe` keyword
- Use `fixed` when pointers reference managed objects

2. Safety:

- These features bypass .NET safety checks
- Use only when absolutely necessary
- Validate all pointer operations

3. P/Invoke Tips:

- Match native types precisely (`int` vs `Int32`)
- Consider marshaling for complex types

- Use `CharSet` in `DllImport` for string handling

MSDN References

- [Unsafe Code](#)
- [fixed Statement](#)
- [P/Invoke](#)